Santa's Agony

Kim Jae Won

Cha Won Woo

Choi Sun

Contents

- Concept
- Walkthrough Example by Demonstrating Algorithms
- Code
- Assigning Tasks

Concept of Santa's Agony

Santa does shopping for the gift.

Santa use Knapsack algorithms to choose a gift for the children.

After that Santa goes around the children's houses and distributes gifts.

At this time, gifts are distributed using Travel Salesman Algorithms.

Shopping - Knapsack Algorithms

We used greedy method in the Knapsack Algorithms.

When shopping, choose gifts based on the "Shopping List" file.

The "Shopping List" file contains 3 types of information - price, number of items, and weight of the items.

When Knapsack Algorithms is executed, worth = price*number/weight.

In addition, capacity is based on weight.

Therefore, the higher the price and number, the smaller the weight, the higher the worth of the item.

Distributing - Travel Salesman Algorithms

We made 'TSP Algorithm' based on "Dynamic Programming".

When distributing presents using TSP, consider the List "good_child_list".

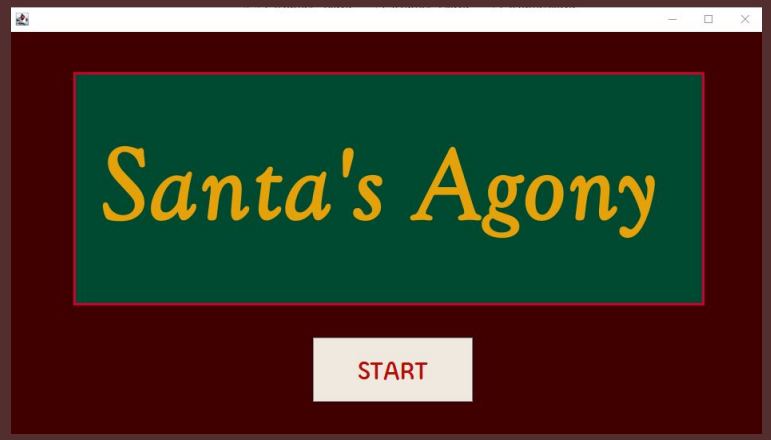
Find the best traversing route considering the nice child point and the distance to the children's house.

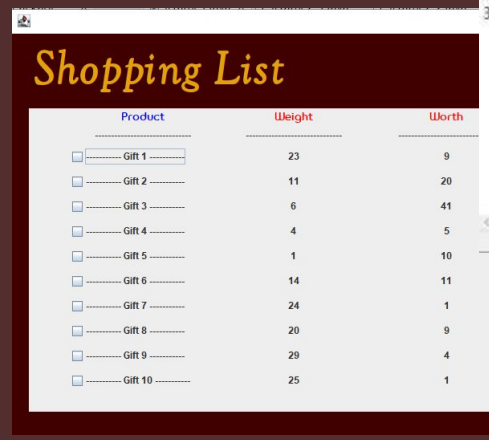
"good_child_list" -> Contains 'Name', 'Location', 'Nice Child Point'

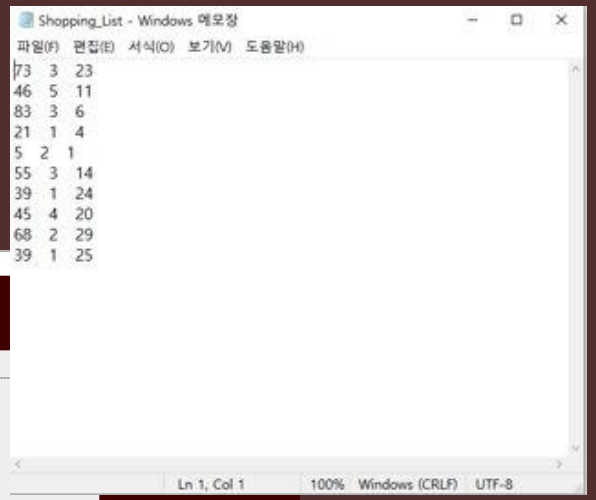
When using TSP Algorithm, we decide cost = (distance)^2 / (Nice Child Point)^2

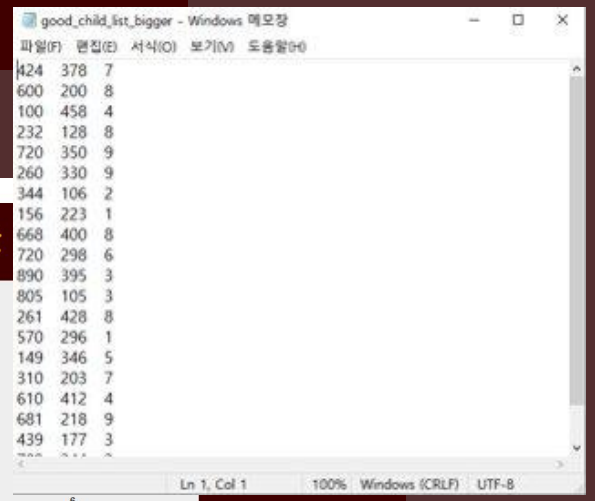
So, the childs who have 'short distance', 'high Nice Child Point' get PRIORITY in selecting.

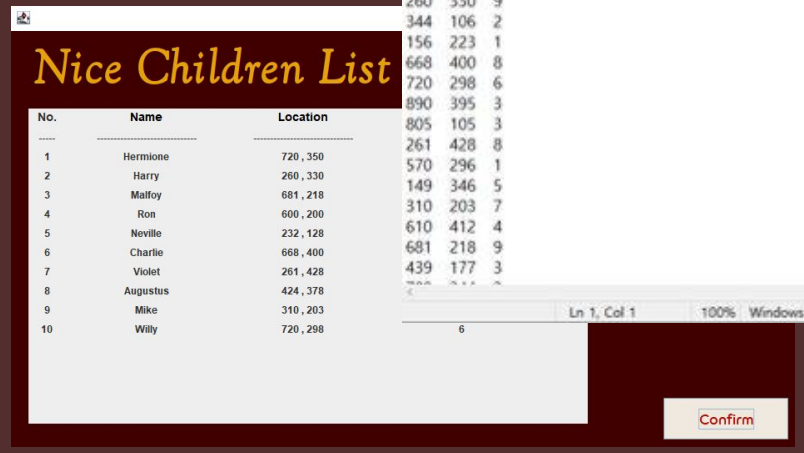
Walkthrough Example











Walkthrough Example

Code – Knapsack

```java
public class Knapsack_Update {
    public static List<Integer> worth_List = new ArrayList<Integer>();
    public static List<Integer> weight_List = new ArrayList<Integer>();
    public static int numberOfItems;

    Knapsack_Update(){
        try {
            worth_List.clear();
            weight_List.clear();

            List<String> allLines = Files.readAllLines(Paths.get("Shopping_List.txt"));
            for (String line : allLines) {
                String[] factor = line.split("   ");
                worth_List.add(Integer.parseInt(factor[0]) * Integer.parseInt(factor[1]) / Integer.parseInt(factor[2]));
                weight_List.add(Integer.parseInt(factor[2]));
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
        int capacity = 150;
        numberOfItems = knapsack(capacity, weight_List, worth_List);
    }
```

Knapsack 클래스 constructor

Knapsack 클래스 method

```java
public static int knapsack(int capacity, List<Integer>  weight_list, List<Integer> worth_list) {

    if (weight_list == null || worth_list == null || weight_list.size() != worth_list.size() || capacity < 0)
        throw new IllegalArgumentException("Invalid input");

    final int N = weight_list.size();
    int[][] DP = new int[N + 1][capacity + 1];

    for (int i = 1; i <= N; i++) {
        int w = weight_list.get(i-1), v = worth_list.get(i-1);
        for (int sz = 1; sz <= capacity; sz++) {
            DP[i][sz] = DP[i - 1][sz];

            if (sz >= w && DP[i - 1][sz - w] + v > DP[i][sz])
                DP[i][sz] = DP[i - 1][sz - w] + v;
        }
    }
    int sz = capacity;
    List<Integer> itemsSelected = new ArrayList<>();
    itemsSelected.clear();
    for (int i = N; i > 0; i--) {
        if (DP[i][sz] != DP[i - 1][sz]) {
            int itemIndex = i - 1;
            itemsSelected.add(itemIndex);
            sz -= weight_list.get(itemIndex);
        }
    }
    java.util.Collections.reverse(itemsSelected);
    return itemsSelected.size();
}
```

Code – Travelling Salesman Problem

```java
public class tsp extends Knapsack_Update{

  public final int N;
  public final int MAX = 30;
  public static int n;
  private final int start;
  private final double[][] distance;
  public List<Integer> tour = new ArrayList<>();
  public static struct[] list;
  private double minTourCost = Double.POSITIVE_INFINITY;
  private boolean ranSolver = false;

  tsp(int type){
      super();
      list = new struct[MAX];
      int x, y, point;

      try{
          String fileName = "good_child_list_bigger.txt";
          FileReader filereader = new FileReader(fileName);
          BufferedReader br = new BufferedReader(filereader);
          String line = "";
          int a = 1;
          while(true) {
              line = br.readLine();
              String[] tokens = line.split("   ");
              x = Integer.parseInt(tokens[0]);
              y = Integer.parseInt(tokens[1]);
              point = Integer.parseInt(tokens[2]);
              if(((x==(int)935/2) && (y==(int)414/2)) || point == 0) {
                  break;
              }
              list[a] = new struct(x, y, point);
              a++;
          }

          br.close();

          if(type==1) {
              n = numberOfItems+1;
          }
          else {
              n=a;
          }

        } catch (FileNotFoundException e){
            System.out.println("Error : File not found");
            e.getStackTrace();
        } catch (Exception e) {
            System.out.println("Error!\n");
            e.printStackTrace();
```

TSP 클래스 constructor 일부

```java
// Solves the traveling salesman problem and caches solution.
public void solve() {

  if (ranSolver) return;      //꼭 해줘야하나?

  final int END_STATE = (1 << N) - 1;
  Double[][] memo = new Double[N][1 << N];

  // Add all outgoing edges from the starting node to memo table.
  for (int end = 0; end < N; end++) {
    if (end == start) continue;
    memo[end][(1 << start) | (1 << end)] = distance[start][end];
  }

  for (int r = 3; r <= N; r++) {
    for (int subset : combinations(r, N)) {
      if (notIn(start, subset)) continue;
      for (int next = 0; next < N; next++) {
        if (next == start || notIn(next, subset)) continue;
        int subsetWithoutNext = subset ^ (1 << next);
        double minDist = Double.POSITIVE_INFINITY;
        for (int end = 0; end < N; end++) {
          if (end == start || end == next || notIn(end, subset)) continue;
          double newDistance = memo[end][subsetWithoutNext] + distance[end][next];
          if (newDistance < minDist) {
            minDist = newDistance;
          }
        }
        memo[next][subset] = minDist;
      }
    }
  }
```

TSP 클래스 method

```java
  // Connect tour back to starting node and minimize cost.
  for (int i = 0; i < N; i++) {
    if (i == start) continue;
    double tourCost = memo[i][END_STATE] + distance[i][start];
    if (tourCost < minTourCost) {
      minTourCost = tourCost;
    }
  }

  int lastIndex = start;
  int state = END_STATE;
  tour.add(start);

  // Reconstruct TSP path from memo table.
  for (int i = 1; i < N; i++) {

    int bestIndex = -1;
    double bestDist = Double.POSITIVE_INFINITY;
    for (int j = 0; j < N; j++) {
      if (j == start || notIn(j, state)) continue;
      double newDist = memo[j][state] + distance[j][lastIndex];
      if (newDist < bestDist) {
        bestIndex = j;
        bestDist = newDist;
      }
    }

    tour.add(bestIndex);
    state = state ^ (1 << bestIndex);
    lastIndex = bestIndex;
  }

  tour.add(start);
  Collections.reverse(tour);

  ranSolver = true;
}
```

Assigning project tasks

PPT, Report: Cha Won Woo, Kim Jae Won

CodingCLI / Making GUI: Choi Sun, Cha Won Woo, Kim Jae Won

Presentation: Choi Sun

Thank you